

LABORATORIO 4

MPI: ¿Más procesos significan siempre mayor velocidad?

Multiplicación matriz-vector y ordenamiento Odd-Even en un clúster MPI

1. Propósito

Estudiar cómo diseñar, ejecutar y evaluar algoritmos paralelos con MPI sobre el clúster configurado previamente. Se trabajará con dos problemas de características diferentes: multiplicación matriz-vector y ordenamiento Odd-Even. El objetivo no es solamente obtener programas correctos, sino analizar cómo la comunicación, sincronización, balance de carga y tamaño del problema afectan el rendimiento.

2. Objetivos

- Implementar una versión secuencial y una versión paralela con MPI de la multiplicación matriz-vector.
- Utilizar operaciones colectivas de MPI para distribuir datos y reunir resultados.
- Implementar o adaptar el ordenamiento Odd-Even utilizando MPI.
- Utilizar comunicación entre procesos vecinos y analizar su costo.
- Ejecutar los programas con diferentes cantidades de procesos y, cuando sea posible, en diferentes cantidades de nodos.
- Medir tiempo de ejecución, speedup y eficiencia.
- Analizar por qué aumentar el número de procesos no siempre produce una aceleración proporcional.

3. Problemas a resolver

3.1 Multiplicación matriz-vector

Dada una matriz A de tamaño $N \times N$ y un vector x de tamaño N , calcular:

$$y = A \cdot x$$

Distribuir las filas de A entre los procesos, de modo que cada proceso calcule una parte de y .

3.2 Ordenamiento Odd-Even

Implementar el ordenamiento Odd-Even Transposition Sort de manera paralela. Cada proceso trabajará con una parte de los datos y, en cada fase, intercambiará información con un proceso vecino cuando corresponda.

4. Parte A — Versión secuencial

Desarrollar o utilizar una versión secuencial de cada problema antes de paralelizar.

- Multiplicación matriz-vector.
- Ordenamiento Odd-Even.
- Verificar la correctitud de ambas soluciones.
- Medir el tiempo de ejecución para diferentes tamaños de entrada.

Utilizar al menos tres tamaños de problema suficientemente grandes para que el tiempo de ejecución sea medible.

5. Parte B — MPI: multiplicación matriz-vector

Implementar la versión MPI distribuyendo las filas de la matriz entre los procesos.

- Inicializar MPI y obtener MPI_Comm_rank y MPI_Comm_size.
- Distribuir las filas de la matriz entre los procesos.
- Distribuir el vector x para que todos los procesos puedan utilizarlo.
- Realizar el producto local matriz-vector.
- Reunir los resultados parciales para formar el vector y.
- Verificar que el resultado MPI coincide con el resultado secuencial.

Se recomienda utilizar MPI_Bcast, MPI_Scatter/MPI_Scatterv y MPI_Gather/MPI_Gatherv, según el diseño elegido.

6. Parte C — MPI: Odd-Even Sort

Implementar o adaptar el algoritmo Odd-Even para ejecutarlo con varios procesos MPI.

- Distribuir los datos entre los procesos.
- Ordenar localmente los datos de cada proceso.
- Realizar las fases de comunicación entre procesos vecinos.
- Comparar e intercambiar los elementos necesarios entre procesos.
- Mantener la sincronización requerida por el algoritmo.
- Construir el resultado global ordenado.
- Verificar que el resultado final está correctamente ordenado.

Analizar el uso de MPI_Send, MPI_Recv o MPI_Sendrecv y justificar la estrategia de comunicación utilizada.

7. Parte D — Correctitud

Para ambos algoritmos:

- Ejecutar con un solo proceso.
- Ejecutar posteriormente con varios procesos.
- Comparar los resultados con la versión secuencial.
- Incluir al menos un caso pequeño cuyo resultado pueda verificarse manualmente.

8. Parte E — Evaluación de rendimiento

Realizar experimentos con diferentes cantidades de procesos. Se recomienda utilizar 1, 2, 4, 8 y 10 procesos, siempre que la infraestructura lo permita.

Registrar: tamaño del problema, número de procesos, número de nodos, tiempo de ejecución y resultado de la ejecución.

9. Métricas

Calcular:

Speedup: $S_p = T_1 / T_p$

Eficiencia: $E_p = S_p / p$

Donde T_1 es el tiempo con un proceso, T_p el tiempo con p procesos y p la cantidad de procesos.

10. Parte F — Ejecución en el clúster

Cuando sea posible, realizar experimentos con:

- Varios procesos dentro de un mismo nodo.
- Procesos distribuidos en dos o más nodos.
- Diferentes cantidades de procesos manteniendo constante el tamaño del problema.

Registrar claramente qué procesos se ejecutaron en qué nodos. El objetivo es observar el efecto de la comunicación distribuida, no solamente comprobar que MPI funciona.

11. Parte G — Análisis comparativo

Responder, como mínimo:

1. ¿Cuál de los dos algoritmos obtiene mayor speedup al aumentar el número de procesos? ¿Por qué?
2. ¿En cuál de los algoritmos tiene mayor importancia la comunicación entre procesos?
3. ¿Qué efecto tiene la sincronización sobre el rendimiento de Odd-Even?
4. ¿Qué ocurre con la eficiencia cuando se incrementa el número de procesos?
5. ¿Por qué utilizar más procesos no garantiza obtener un speedup proporcional?
6. ¿Qué ocurre cuando el tamaño del problema es pequeño? ¿Y cuando es grande?
7. ¿Se observa diferencia entre ejecutar varios procesos en un mismo nodo y distribuirlos entre varios nodos?
8. ¿Qué relación existe entre tiempo de cómputo, comunicación y latencia?
9. ¿Qué posibles problemas de balance de carga pueden aparecer?
10. ¿En qué situación recomendaría utilizar la versión paralela y en cuál la secuencial?

12. Parte H — Experimento de escalabilidad

Seleccione uno de los dos algoritmos y mantenga fijo el tamaño del problema. Ejecute con el mayor número de configuraciones de procesos que permita el clúster.

Construya tablas y gráficas de: procesos vs. tiempo; procesos vs. speedup; procesos vs. eficiencia.

13. Entregable

- Código fuente de las versiones secuencial y MPI de ambos problemas.
- Instrucciones para compilar y ejecutar.
- Tabla de resultados experimentales.
- Cálculo de speedup y eficiencia.
- Gráficas de rendimiento.

- Descripción de la configuración utilizada del clúster.
- Respuestas justificadas a las preguntas de análisis.
- Conclusiones.

14. Recomendaciones

- Se permite utilizar como punto de partida los ejemplos de multiplicación matriz-vector y ordenamiento paralelo del libro.
- La reproducción del ejemplo no constituye por sí sola el objetivo: debe adaptarse, ejecutarse en el clúster y evaluarse experimentalmente.
- No comparar ejecuciones con tamaños diferentes cuando se quiera estudiar exclusivamente el efecto del número de procesos.
- Realizar varias mediciones cuando sea necesario y utilizar una metodología consistente.
- No asumir que una ejecución más rápida implica necesariamente un mejor algoritmo: analizar también speedup y eficiencia.
- Verificar siempre la correctitud antes de analizar el rendimiento.

15. Idea central del laboratorio

La pregunta que debe guiar el laboratorio es: “¿Qué ocurre realmente cuando agregamos más procesos?”

La respuesta debe construirse a partir de los experimentos, relacionando el rendimiento con la cantidad de cómputo, comunicación, sincronización, balance de carga, tamaño del problema y arquitectura del clúster.